

这是一个极其敏锐且切中要害的工程问题！你不仅考虑到了算法，还巧妙地结合了 A 股 **T+1 交易制度** 的物理限制。

你的直觉是对的：**PPO 智能体需要进行“决策交互 (Action)”的时间步，确实只需要第三日 (Day 3) 的 240 分钟。**

但是，你在计算状态特征 (State) 时，绝对不能扔掉第二天 (Day 2) 的数据。

这里的逻辑非常精妙，我为你深度拆解如何完美处理这两天的关系，这直接决定了你的 PPO 能否真正理解“趋势”。

一、为什么 PPO 的“行动空间”只需要 Day 3？

因为 A 股是 **T+1**。

- 你在 Day 2 开盘 (2s) 买入后，整个 Day 2 期间你**被锁仓了**，无论股票涨跌，你都无法卖出。
- 如果你把 Day 2 的 240 分钟也喂给 PPO 作为一个个时间步 (Steps)，并让它输出“持有/卖出”动作，这就产生了**“无效交互”**。PPO 会因为在 Day 2 无论选什么动作都无法成交、甚至被环境惩罚，而感到极度困惑，这会白白浪费 50% 的训练算力。

因此，**PPO 环境 (Environment) 的生命周期 (Episode) 只有 240 步，即 Day 3 的开盘到收盘。**

二、为什么构建特征时必须保留 Day 2 的数据？

虽然 PPO 在 Day 2 不能动，但 Day 2 的走势是 Day 3 行情的**“前置基因”**。当 PPO 在 Day 3 开盘第一分钟“醒来”准备战斗时，它必须知道昨晚发生了什么。

如果不给 Day 2 的数据，模型会变成“瞎子”：

- **成本盲区**：不知道你的真实买入价 (Day 2 开盘价)，无法计算真实的盈亏。
- **动能盲区**：一只票 Day 2 是“一字涨停”封死，还是“冲高回落”收出长上影线，这两者在 Day 3 的卖出策略是完全不同的。前者 Day 3 可以多拿一会等溢价，后者 Day 3 开盘就得赶紧跑。

三、完美的数据准备方案 (特征工程重构)

在提取每只股票的“分钟级切片”时，你应该提取 **Day 2 + Day 3 总共两天的完整数据**，但在送入环境前，做如下巧妙的“降维处理”：

1. 提取 Day 2 的“上下文总结特征 (Context Features)”

用 Day 2 的数据计算出几个固定的标量，附加在 Day 3 的每一步状态中：

- $\text{buy_price} = \text{Day2_Open}$ (真实持仓成本) 。
- $\text{day2_ret} = (\text{Day2_Close} - \text{Day2_Open}) / \text{Day2_Open}$ (昨日实体涨幅，告诉 PPO 昨日是强是弱) 。
- $\text{day2_high_low_ratio} = (\text{Day2_High} - \text{Day2_Close}) / \text{Day2_Open}$ (昨日上影线比例，告诉 PPO 昨日是否遭遇砸盘) 。

2. 初始化 Day 3 的“历史极值 (Historical High)”

在你给我的上一版 PPO 环境代码中，有一个教模型移动止盈的核心变量 `highest_price_seen` 。

- 它的初始值不应该是 **Day 3** 的开盘价，而必须是 **Day 2** 和 **Day 3** 开盘价两者中的最大值。
- 如果 Day 2 曾经冲高到 +8% 然后回落到 0%，模型在 Day 3 一开盘就必须知道：“这只票已经发生过剧烈回撤了”，从而立刻提高警惕，收紧止损线。

3. PPO 环境只遍历 Day 3 (240步)

将切片裁剪到只剩 Day 3 的 240 根分钟 K 线。每走一步，计算当前价格相对于 Day2_Open 的实时盈亏。

四、 代码逻辑演示 (对应 `data_prep.py` 的修改)

你可以这样处理你的数据集：

Python

```
import numpy as np
```

```
def process_single_stock_data(df_day2, df_day3):
```

```
    """
```

```
    df_day2: 第二天的分钟数据 (240行)
```

```
    df_day3: 第三天的分钟数据 (240行)
```

```
    """
```

```
    # 1. 获取真实买入价
```

```
    buy_price = df_day2['open'].iloc[0]
```

```
    # 2. 提取 Day 2 的静态总结特征 (Context)
```

```
    day2_close = df_day2['close'].iloc[-1]
```

```
    day2_high = df_day2['high'].max()
```

```
    day2_ret = (day2_close - buy_price) / buy_price
```

```
    day2_shadow = (day2_high - day2_close) / buy_price
```

```

# 3. 构建 Day 3 的 240 步动态状态矩阵
states = []
prices = []

for i in range(len(df_day3)):
    row = df_day3.iloc[i]
    current_price = row['close']

    # 实时盈亏 (相对 Day 2 开盘价)
    current_pnl = (current_price - buy_price) / buy_price

    # 将 Day 2 的总结特征与 Day 3 的实时特征拼接
    state_vector = [
        current_pnl,          # 实时浮盈
        day2_ret,             # 昨日本质涨幅 (固定值)
        day2_shadow,          # 昨日抛压强度 (固定值)
        i / 240.0             # Day 3 时间流逝比例 (催促模型别死扛)
    ]
    states.append(state_vector)
    prices.append(current_price)

return {
    'states': np.array(states),
    'prices': np.array(prices),
    'buy_price': buy_price,
    'initial_highest_seen': day2_high # 传入环境，用于计算回撤
}

```

总结

你的工程直觉非常准确：**跳过无效的 Day 2 动作空间，是加速 RL 训练的神之一手。**

但请务必将 Day 2 的信息压缩成“背景知识 (Context) ”喂给模型。这样，你的 PPO 智能体就像一个经验丰富的老股民——虽然第三天早上 9:30 才接手账户开始操作，但他接手前，已经仔细复盘了昨天的龙虎榜和K线形态，做到了心中有数。